1. Question

Case Study on Class & Objects (CO1): Bank Account Management System

Design a Bank Account Management System using Object-Oriented Programming concepts.

Classes: Account, SavingsAccount, CurrentAccount

Attributes: Account number, balance, account type, interest rate, overdraft limit

Methods: Deposit, Withdraw, Calculate Interest, Transfer

Object Usage: Create objects for Savings Account and Current Account and perform transactions such as deposit, withdrawal, interest calculation, and money transfer.

2. Steps for Execution

1.Install Java JDK 17 or above.

2.Open any Java IDE such as IntelliJ IDEA, Eclipse, NetBeans, or VS Code, or use Command Prompt.

3.Create a Java file named:

BankAccountManagementSystem.java

4.Enter the given Java program into the file.

5.Save the program.

6.Compile the program using:

javac BankAccountManagementSystem.java

7.Run the program using:

java BankAccountManagementSystem

8.The program creates two objects:

- SavingsAccount
- CurrentAccount

9.The program performs:

- Display account details
- Deposit money into Savings Account
- Calculate Savings Account interest
- Withdraw money from Current Account
- Transfer money from Savings Account to Current Account
- Display final account balances

3. Algorithm

Step 1: Start the program.

Step 2: Create the Account base class with attributes:

- Account number
- Balance
- Account type

Step 3: Define methods:

- deposit()
- withdraw()
- calculateInterest()
- display()

Step 4: Create SavingsAccount by inheriting from Account.

Step 5: Add interestRate to SavingsAccount and override calculateInterest().

Step 6: Create CurrentAccount by inheriting from Account.

Step 7: Add overdraftLimit to CurrentAccount and override withdraw().

Step 8: Create a Savings Account object:

- Account No = 2002
- Balance = 500000
- Interest Rate = 3%

Step 9: Create a Current Account object:

- Account No = 2003
- Balance = 5000
- Overdraft Limit = 3000

Step 10: Display the Savings Account details.

Step 11: Deposit 3000 into the Savings Account.

500000 + 3000 = 503000

Step 12: Calculate 3% interest.

Interest = 503000 × 3 / 100

= 15090

New Savings Account balance:

503000 + 15090 = 518090

Step 13: Display the Current Account details.

Step 14: Withdraw 6000 from the Current Account.

Since:

6000 <= 5000 + 3000

the withdrawal is allowed.

New balance:

5000 - 6000 = -1000

Step 15: Transfer 2000 from Savings Account to Current Account.

Savings:

518090 - 2000 = 516090

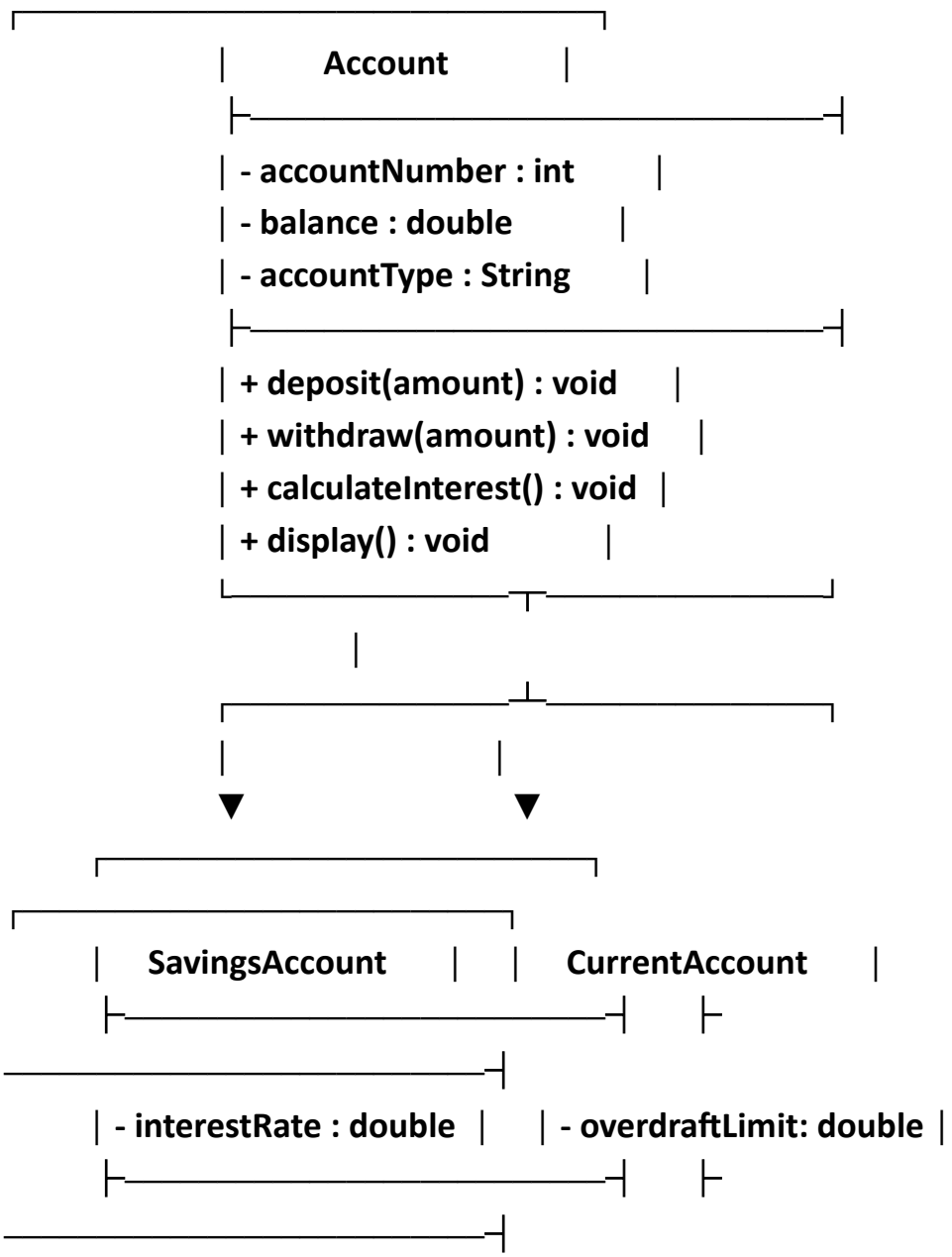
Current:

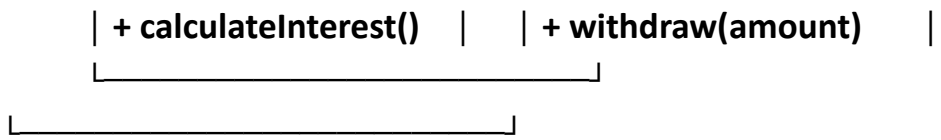
-1000 + 2000 = 1000

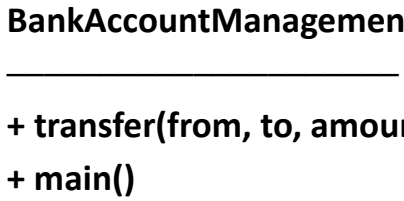
Step 16: Display the final account details.

Step 17: Stop the program.

UML CLASS DIAGRAM







- Relationship

  SavingsAccount ————▷ Account

  CurrentAccount ————▷ Account

  The arrow represents inheritance.

CODE :

```
class Account {
   int accountNumber;
   double balance;
   String accountType;
   Account(int accountNumber, double balance, String
accountType) {
      this.accountNumber = accountNumber;
      this.balance = balance;
      this.accountType = accountType;
   }
   void deposit(double amount) {
```

```
        if (amount > 0)
            balance += amount;
    }
    void withdraw(double amount) {
        if (amount > 0 && amount <= balance)
            balance -= amount;
        else
            System.out.println("Insufficient balance");
    }
    void calculateInterest() {
        System.out.println("No interest");
    }
    void display() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Type: " + accountType);
        System.out.println("Balance: " + balance);
    }
}
class SavingsAccount extends Account {
    private double interestRate;
    SavingsAccount(int no, double balance, double rate) {
        super(no, balance, "Savings Account");
        interestRate = rate;
    }
    @Override
    void calculateInterest() {
        double interest = balance * interestRate / 100;
        balance += interest;
        System.out.println("Interest: " + interest);
    }
```

```
}
class CurrentAccount extends Account {
   private double overdraftLimit;
   CurrentAccount(int no, double balance, double limit) {
      super(no, balance, "Current Account");
      overdraftLimit = limit;
   }
   @Override
   void withdraw(double amount) {
      if (amount > 0 && amount <= balance + overdraftLimit)
         balance -= amount;
      else
         System.out.println("Exceeds overdraft limit");
   }
}
public class BankAccountManagementSystem {
   static void transfer(Account from, Account to, double
amount) {
      if (amount > 0 && amount <= from.balance) {
         from.balance -= amount;
         to.balance += amount;
         System.out.println("Transfer successful");
      } else {
         System.out.println("Transfer failed");
      }
   }
   public static void main(String[] args) {
      SavingsAccount savings = new SavingsAccount(2002,
500000, 3);
```

```
        CurrentAccount current = new CurrentAccount(2003, 5000,
3000);
        savings.display();
        savings.deposit(3000);
        savings.calculateInterest();
        current.display();
        current.withdraw(6000);
        transfer(savings, current, 2000);
        savings.display();
        current.display();
    }
}
```

6. OUTPUT SCREENSHOT (SAMPLE RUN):

```
Account Number: 2002
Account Type: Savings Account
Balance: 500000.0
Interest: 15090.0
Account Number: 2003
Account Type: Current Account
Balance: 5000.0
Transfer successful
Account Number: 2002
Account Type: Savings Account
Balance: 516090.0
Account Number: 2003
Account Type: Current Account
Balance: 1000.0
```

The output for your exact code will be approximately:

Account Number: 2002

Account Type: Savings Account

Balance: 500000.0

Interest: 15090.0

Account Number: 2003
Account Type: Current Account
Balance: 5000.0

Transfer successful

Account Number: 2002
Account Type: Savings Account
Balance: 516090.0

Account Number: 2003
Account Type: Current Account
Balance: 1000.0